

Tema 3

Conceptos avanzados de tipos y variables



Índice

- Introducción
- Tipos de valor
- Tipos de referencia
- Tipos de estructura
- Tipos de enumeración
- *null*
- Inmutabilidad



- En el tema anterior vimos qué son las variables y su importancia en la programación.
- También conocimos los tipos de datos primitivos de C# y otro tipo de datos para trabajar con cadenas (`string`).
- En este tema aprenderemos conceptos teóricos sobre los tipos de datos y las variables y conoceremos cómo estos nos afectan a la hora de programar.

1. Introducción



- Como vimos en el primer tema, una **variable** se define como un espacio de memoria en el que se guarda un dato.
- Cada variable tiene asignado un tipo de dato, que es único e inmutable.
- En C#, todos los tipos de datos derivan de la clase **object**, lo que nos proporciona unos mecanismos que comparten todos los tipos.

1. Introducción

Métodos de la clase `Object`

- `obj.Equals(other)`: Devuelve `true` si el objeto en el que se invoca el método (`obj`) es igual al objeto pasado como parámetro (`other`).
- `obj.ToString()`: Devuelve una cadena con la representación en texto del objeto `obj`. Este método se llama implícitamente si concatenamos cadenas con objetos.

1. Introducción

Métodos de la clase `Object`

- `obj.GetType()`: Devuelve el tipo de objeto (clase) al que pertenece `obj`.
- `obj.GetHashCode()`: Devuelve un código numérico único (hash) que representa un objeto y su estado particular. Siempre que el estado no cambie, debe devolver el mismo valor, mientras que si el objeto cambia de estado, debe devolver un valor distinto.

1. Introducción

- Los tipos de valor son aquellos en los que el espacio de memoria de la variable contiene directamente el **dato o valor en sí mismo**.
- Son tipos de valor todos los tipos básicos primitivos que vimos en el primer tema:
 - Tipos numéricos enteros
 - Tipos numéricos de punto flotante
 - `bool`
 - `char`

2. Tipos de valor

Copias de tipos de valor

- Al asignar una variable de tipo de valor a otra, se crea una **copia** del valor en la memoria correspondiente a la variable de destino. Ocurre lo mismo si se pasa como parámetro o si es un valor devuelto por un método (veremos que es esto en el tema 5).
- Esto supone que si copiamos una variable de tipo de valor a otra y modificamos la segunda, esta modificación **no afecta a la primera**.

2. Tipos de valor

Ejemplo

- La modificación de y no afecta al contenido de x ya que se ha creado una copia independiente de su valor.

2. Tipos de valor

```
1 int x = 6;  
2 int y = x;  
3 y = 85;  
4  
5 Console.WriteLine("La variable x e
```



Comparación de tipos de valor

- La comparación de variables de tipo de valor (ya sea mediante el método `Equals` o con el operador `==`) compara el **valor** contenido en las variables.
- Esto quiere decir que la comparativa será `true` si el **dato** contenido en una variable es igual al dato contenido en la otra variable.

2. Tipos de valor

Ejemplo

- Aunque son dos variables distintas, la comparativa en los tipos de valor se hace a nivel de dato contenido, por lo que en este caso `radius` y `height` son iguales.

2. Tipos de valor

```
1 double radius = 7.6;  
2 double height = 7.6;  
3  
4 Console.WriteLine(radius == height);  
5 // Imprime "True"  
6 Console.WriteLine(radius.Equals(height));  
7 // Imprime "True"
```

ToString

- El método `ToString()` en los tipos de valor devuelve una cadena con el dato contenido en la variable en forma de cadena de texto.
- Si trabajamos con cadenas, el método se llama implícitamente aunque únicamente pongamos el nombre de la variable:

2. Tipos de valor

```
1  double price = 48.99;  
2  
3  Console.WriteLine(price.ToString());  
4  Console.WriteLine(price);  
5  // Ambas sentencias son equivalentes. Muestran por pa  
6  
7  Console.WriteLine("El precio del producto es: " + pri  
8  Console.WriteLine("El precio del producto es: " + pri  
9  // Ambas sentencias son equivalentes.
```

Asignación

2. Tipos de valor



Copia

2. Tipos de valor



Modificación

2. Tipos de valor

Vídeo práctico

2. Tipos de valor

Vídeo práctico - código

2. Tipos de valor

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             long x = 481084;
8             float height = 8.581f;
9             char firstCharacter = 'J';
10            bool isEnabled = true;
11
12            Console.WriteLine("x es " + x);
13            Console.WriteLine(string.Format("height es
14
15            long y = x;
16            y = -180472;
17
18            char secondCharacter = firstCharacter;
19            secondCharacter = 'ñ';
20
21            Console.WriteLine(string.Format("x: {0}, y
22
```

- Los tipos de referencia son aquellos en los que el espacio de memoria de la variable contiene una **referencia** al espacio de memoria donde realmente se almacena el dato.
- En C#, la **gran mayoría** de tipos son tipos de referencia. Todos los tipos personalizados lo son y, en general, prácticamente cualquier tipo que esté fuera de los tipos de datos primitivos, los tipos de estructura o los tipos de enumeración.

3. Tipos de referencia

Copias de tipos de referencia

- Al asignar una variable de tipo de referencia a otra, se copia a la nueva variable la **dirección de memoria** donde se encuentran los datos (referencia).
- Por tanto, si copiamos una variable de tipo de referencia a otra y la modificamos, estas modificaciones afectarán **tanto a la copia como a la original**.

3. Tipos de referencia

3. Tipos de referencia

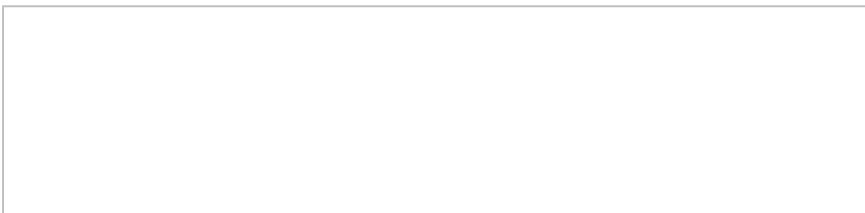
```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Animal dog = new Animal("Perro", 5);
8             Animal squirrel = dog;
9
10            squirrel.Species = "Ardilla";
11
12            Console.WriteLine("La especie del animal 'perro' es:");
13            Console.WriteLine("La especie del animal 'ardilla' es:");
14        }
15    }
16
17    public class Animal
18    {
19        public Animal(string species, int age)
20        {
21            Species = species;
22            Age = age;
23        }
24
25        public string Species { get; set; }
26        public int Age { get; set; }
27    }
28 }
```

Ejemplo

Ejemplo

- En este caso, la modificación de una variable de tipo de referencia copiada, resulta en la modificación de la variable original desde la que se copió.

3. Tipos de referencia



Comparación de tipos de referencia

- La comparación de variables de tipo de referencia (ya sea mediante el método `Equals` o con el operador `==`) compara la **referencia** a la que apunta la memoria.
- Esto quiere decir que la comparativa será `true` únicamente si ambas variables apuntan a la misma referencia.
- Este comportamiento lo podremos sobrescribir en las clases que escribamos nosotros.

3. Tipos de referencia

3. Tipos de referencia

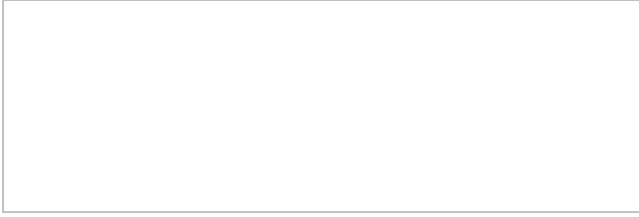
```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Animal dog1 = new Animal("Perro", 1);
8             Animal dog2 = new Animal("Perro", 1);
9             Animal dog3 = dog1;
10
11             Console.WriteLine(dog1 == dog2);
12             Console.WriteLine(dog1.Equals(dog3));
13         }
14     }
15
16     public class Animal
17     {
18         public Animal(string species, int age)
19         {
20             Species = species;
21             Age = age;
22         }
23
24         public string Species { get; set; }
25         public int Age { get; set; }
26     }
```

Ejemplo

3. Tipos de referencia

Ejemplo

- En este caso, la primera comparativa (entre perro1 y perro2) es false ya que a pesar de contener los mismos datos, las referencias de las variables son distintas.
- La comparativa entre perro1 y perro3 es true porque se ha copiado la **referencia** de la primera en la segunda.



ToString

- El método `ToString()` en los tipos de referencia devuelve por defecto una cadena con la **identificación del tipo** del objeto.
- **No** tenemos información del dato contenido en esta cadena.
- Podemos sobrescribir este método en nuestras clases personalizadas para obtener cadenas que nos aporten más información.

3. Tipos de referencia

Ejemplo

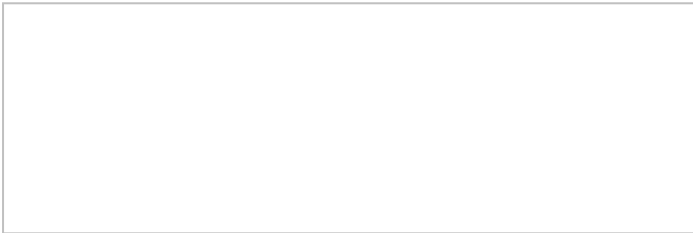
3. Tipos de referencia

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Animal horse = new Animal("Caballo", 4);
8             Animal bear = new Animal("Oso", 8);
9
10            Console.WriteLine(horse);
11            Console.WriteLine(bear);
12        }
13    }
14
15    public class Animal
16    {
17        public Animal(string species, int age)
18        {
19            Species = species;
20            Age = age;
21        }
22
23        public string Species { get; set; }
```

Ejemplo

- En el ejemplo anterior, el método `ToString` únicamente devuelve información del tipo de dato (clase). **NO devuelve información del valor** contenido en las variables.

3. Tipos de referencia



Caso particular del tipo `string`

- En C#, el tipo `string` es un tipo de **referencia**. Sin embargo, muestra un comportamiento especial respecto a lo que hemos visto.
- Tanto en la copia de variables de tipo `string` como en su comparación, se comportan de una forma más parecida a los tipos de valor debido a que se ha **sobrescrito** el comportamiento de estas operaciones.

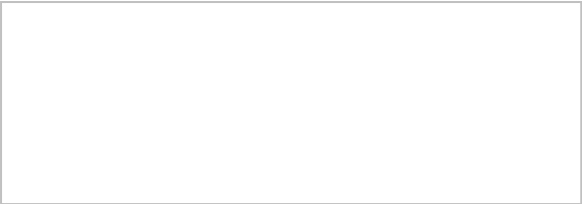
3. Tipos de referencia

Ejemplo

- Aunque hemos copiado una variable de tipo de referencia a otra, su modificación no le ha afectado. La clase `string` se comporta de manera parecida a los tipos de valor ante la copia y modificación.

3. Tipos de referencia

```
1 string name = "Antonio"  
2 string surname = name;  
3  
4 surname = "De la Fuente"  
5
```

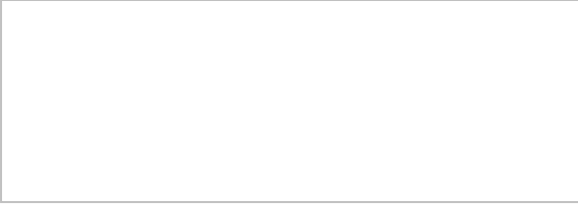


Ejemplo

- Aunque las referencias de ambas variables son distintas, la comparación entre strings también se comporta como de manera similar a la comparación entre tipos de valor.

3. Tipos de referencia

```
1 string name = "Manuel";  
2 string surname = "Manuel";  
3  
4 Console.WriteLine(name == surname)
```



Asignación

3. Tipos de referencia



Copia

3. Tipos de referencia



Modificación

3. Tipos de referencia



Vídeo práctico

3. Tipos de referencia

Vídeo práctico - código

3. Tipos de referencia

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Car focus = new Car("Ford", "Focus", 1999)
8
9             Car auris = focus;
10            auris.Brand = "Toyota";
11            auris.Model = "Auris";
12            auris.Year = 2023;
13
14            Console.WriteLine("focus: {0} {1} ({2})", focus.Brand, focus.Model, focus.Year);
15            Console.WriteLine("auris: {0} {1} ({2})", auris.Brand, auris.Model, auris.Year);
16
17            Car focus2 = new Car("Ford", "Focus", 1999)
18
19            Console.WriteLine("focus y focus2 son iguales");
20        }
21    }
22 }
```

- En C#, un tipo de estructura (struct) es un tipo de valor que agrupa varias variables bajo un único nombre.
- A diferencia de las clases, las estructuras son tipos de valor, por lo que la copia de estructuras replica el valor contenido en vez de copiar una referencia a ellas.
- Nos permiten definir tipos compuestos manteniendo la ligereza de los tipos de valor (más eficientes en memoria).

4. Tipos de estructura

- Estos tipos nos permiten definir tipos de datos personalizados y complejos, pero con un comportamiento más ligero que las clases.
- Con las estructuras podemos definir tipos compuestos manteniendo la ligereza de los tipos de valor (más eficientes en memoria).

4. Tipos de estructura

- A diferencia de las clases, al ser las estructuras tipos de valor, la copia de estructuras replica el valor contenido en vez de duplicar una referencia a ellas.
- Las estructuras **no permiten herencia**. Ni una estructura puede heredar de una clase o estructura, ni otra clase o estructura puede heredar de una estructura. Sí pueden implementar interfaces.
- Los structs en C# pueden contener campos, métodos, constructores y otras estructuras.

4. Tipos de estructura

- Ejemplo

4. Tipos de estructura

```
1 struct Point
2 {
3     double X;
4     double Y;
5
6     Point(double x, double y)
7     {
8         X = x;
9         Y = y;
```

```
10     }  
11  
12     void Move(double xMovement, double yMovement)  
13     {  
14         X += xMovement;  
15         Y += yMovement;  
16     }
```

- En el ejemplo anterior tenemos una estructura que representa un punto con 2 coordenadas de tipo `double`.
- Tiene un constructor para crear variables del tipo `Point` mediante con las coordenadas iniciales especificadas.
- También tiene un método para desplazar las coordenadas el valor indicado.
- Comprenderemos mejor estos conceptos en los próximos temas.

4. Tipos de estructura

Vídeo práctico

4. Tipos de estructura

Vídeo práctico - código

4. Tipos de estructura

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Circle circle1 = new Circle(10, 5.5, 12);
8             Circle circle2 = new Circle(0, 0, 5);
9
10            circle1.PrintArea();
11            circle2.PrintPerimeter();
12
13            Circle circle3 = new Circle(0, 0, 5);
14
15            Console.WriteLine(circle3);
16
17            Circle circle4 = circle1;
18            circle4.Move(5, 4.4);
19
20            Console.WriteLine("circle1 y circle4 son iguales");
21        }
22    }
23 }
```

- En C#, un tipo **enum** es un tipo especial de valor que permite definir un conjunto de valores constantes cada uno con nombre identificativo.
- Los **enumeradores** son una forma útil de organizar valores relacionados de manera simbólica, mejorando la legibilidad y el mantenimiento del código.
- Cada constante se llama **miembro de enumeración** y tiene asignado un valor numérico entero, que por defecto empieza en 0 y se incrementa en 1 para cada miembro. No obstante, es posible especificar el número asociado a cada elemento.

5. Tipos de enumeración

- Sintaxis:

5. Tipos de enumeración

```
1 enum NombreEnumera
2 {
3     Miembro1,
4     Miembro2,
5     Miembro3
6 }
```

- Ejemplo:

- En este ejemplo, el enumerador `Month` contiene 12 miembros (meses del año). A cada miembro se le asigna un valor numérico entero, comenzando por 0 (`January = 0`, `February = 1`, etc.).

5. Tipos de enumeración

```
1  enum Month
2  {
3      January,
4      February,
5      March,
```

6	April,
7	May,
8	June,
9	July,
10	August,
11	September,
12	October,
13	November,

- Como hemos dicho, también podemos personalizar el número asociado a cada uno de los miembros de la enumeración:

5. Tipos de enumeración

```
1 enum HttpStatusCode
2 {
3     BadRequest = 400,
4     AuthRequired = 401,
5     Forbidden = 403,
6     NotFound = 404,
7     UnsupportedMediaType :
8     Locked = 423
9 }
```

```
1 enum DayOfWeek
2 {
3     Monday = 1,
4     Tuesday = 2,
5     Wednesday =
6     Thursday = 4
7     Friday = 5,
8     Saturday = 6
9     Sunday = 7
```

- Podemos acceder al número asignado a un miembro de la enumeración mediante un casting a `int`.
- También podemos hacerlo en sentido inverso, obtener el miembro a partir de un `int`.

5. Tipos de enumeración

```
1 int dayNumber = (int)DayOfWeek.Wednesday;  
2 DayOfWeek day = (DayOfWeek)6;  
3  
4 Console.WriteLine(day);  
5 Console.WriteLine("El número de día del miércoles es: ")
```

- Otra de las ventajas de los tipos `enum` es que su método `ToString` nos devuelve el identificador del miembro.
- En el ejemplo anterior hemos visto como escribía por pantalla "Saturday", ofreciendo información del valor contenido, al contrario de lo que pasa con los tipos de referencia.

5. Tipos de enumeración

Vídeo práctico

5. Tipos de enumeración

Vídeo práctico - código

5. Tipos de enumeración

```

1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("Hello, World!");
8
9             Position playerPosition = Position.Midfielder;
10
11             Console.WriteLine(playerPosition.ToString());
12
13             Position midfielder = Position.Midfielder;
14
15             Console.WriteLine("playerPosition y midfielder so
16
17             Position striker = midfielder;
18             striker = Position.Striker;
19
20             Console.WriteLine("midfielder: " + midfielder + "
21
22             int positionIndex = (int) Enum.Parse(typeof(Position), "Midfielder", true);
23             Console.WriteLine(positionIndex);
24         }
25     }
26 }

```

- En C#, el concepto `null` se refiere a la **ausencia de un valor** en una variable.
- Es decir, `null` indica que una variable no contiene ninguna referencia o, dicho de otra manera, que no apunta a ningún objeto en la memoria.
- Este concepto aplica únicamente a los **tipos de referencia**, ya que los tipos de valor por definición no almacenan referencias sino el dato que contienen directamente.

6. null

- **Siempre que declaramos** una variable de tipo de referencia, su referencia es `null` hasta que se le realice una asignación de su valor.
- Como medida de seguridad, el compilador no nos permite utilizar un tipo de referencia que no ha sido asignado.

6. null

```
1 Animal sheep;  
2 // En este momento, 'sheep' es null  
3  
4 Console.WriteLine(sheep); // ERROR, no compila
```

- Sin embargo, una vez ha sido asignado, aunque sea a `null`, o si la variable procede desde otro ámbito (como un parámetro en un método, lo veremos en el tema siguiente), el compilador NO puede comprobar si es `null`.
- Esto supone un **peligro** a tener en cuenta, ya que intentar acceder a una variable que es `null` provocará una excepción en tiempo de ejecución.

6. null

```
1 Animal sheep = null;  
2 // En este momento, 'sheep' es null  
3  
4 Console.WriteLine(sheep);  
5 // Compila pero no imprime nada
```

```
6  
7 Console.WriteLine(sheep.Species);
```

- En el ejemplo anterior, el programa intentar acceder a la memoria donde se encuentra almacenado el valor de la propiedad `Especies` dentro de nuestra variable.
- Dado que no hay ningún objeto al cual acceder, se produce una **excepción**.
- Es muy importante controlar los posibles objetos null cuando escribamos nuestros métodos y clases.

6. null

- Por ejemplo, un caso típico es la comprobación del valor al principio de un bloque de código antes de hacer operaciones con la variable.
- En este caso, nos protegemos ante una posible variable nula y evitamos que nuestro programa muera inesperadamente.

6. null

```
1 void PrintSpecies(Animal animal)
2 {
3     if(animal == null)
4     {
5         Console.WriteLine("El animal no puede ser null");
6     }
7     else
8     {
```

```
9  
10  
11 }
```

```
Console.WriteLine("El animal es de la especie " +
```

- En el tema 8 veremos el manejo y captura de excepciones, como por ejemplo, la que se produce al intentar acceder a una variable nula.
- Sin embargo, desde el principio es importante tener claro el concepto `null` y cómo este nos afecta si no lo controlamos en nuestro código.

6. null

Vídeo práctico

6. null

Vídeo práctico - código

6. null

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Car ibiza = new Car("Seat", "Ibiza", 2010);
8
9             if (ibiza != null)
10            {
11                Console.WriteLine(ibiza.Brand);
12            }
13        }
14    }
15
16    public class Car
17    {
18        public string Brand;
19        public string Model;
20        public int Year;
21
22        public Car(string brand, string model, int year)
```

- En C#, llamamos **inmutabilidad** a la característica de ciertos tipos de datos que, una vez creados, no pueden ser modificados.
- La inmutabilidad es un concepto importante, ya que puede **mejorar la seguridad** del código y evitar errores difíciles de depurar.

7. Inmutabilidad

- Un **objeto es inmutable** cuando no permite cambiar su valor o estado después de ser creado.
- Cualquier operación que intente modificar un objeto inmutable **creará un nuevo objeto** en lugar de alterar el original.
- Esto significa que todas las propiedades de un objeto inmutable son **solamente de lectura** y se deben inicializar en el momento de la creación del objeto.

7. Inmutabilidad

- Por ejemplo, como vimos en el tema anterior, el tipo `string` en C# es inmutable.
- Esto significa que una vez que se ha creado una cadena, su valor no puede cambiar.
- Cualquier operación que parezca modificar una cadena, en realidad está creando una nueva instancia de cadena.

7. Inmutabilidad

- Ejemplo:

- Aunque nos parezca que la operación += modifica el contenido del objeto hello, realmente se está creando una **nueva instancia** de la clase string y asignando la memoria de la nueva instancia a la variable.

7. Inmutabilidad

```
1 string hello = "Hola";  
2 hello += ", mundo";  
3 // Esto crea una nueva cadena, no modifica  
4  
5 Console.WriteLine(hello);  
6 // Imprime "Hola, mundo"
```

- En C# podemos declarar un campo como inmutable mediante la palabra clave `readonly`.
- Dicha palabra clave significa que solo se les puede asignar un valor durante la inicialización (en el constructor) y no puedan cambiar después.

7. Inmutabilidad

- Si tomamos como ejemplo la clase `Animal` que hemos visto durante este tema:
- Si hacemos `readonly` el campo `Species` únicamente se puede asignar en el constructor y ya no podrá ser modificado posteriormente.

7. Inmutabilidad

```
1 public class Animal
2 {
3     public readonly string Species;
4     public int Age { get; set; }
5 }
```

```
6 public Animal(string species, i  
7 {  
8     Species = species;  
9     Age = age;  
10 }
```

- Teniendo esto en cuenta, podemos crear objetos totalmente inmutables si:
 - Todos sus campos y propiedades son readonlly.
 - Evitamos crear métodos que cambien el estado del objeto.
 - Inicializamos todas las propiedades en el constructor del objeto.

7. Inmutabilidad

- La inmutabilidad es una de las características clave de la **programación funcional** que vimos en el tema 1, por lo que deberemos implementarla cuando queramos poner en práctica dicho paradigma de programación en nuestro código C#.
- Profundizaremos en el uso de la inmutabilidad en el tema 7, que trata sobre clases y objetos.

7. Inmutabilidad

Vídeo práctico

7. Inmutabilidad

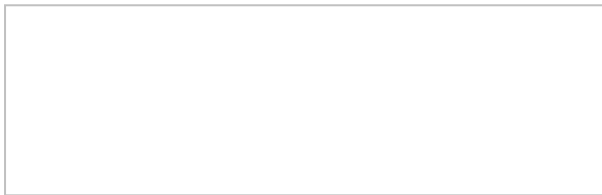
Vídeo práctico - código

7. Inmutabilidad

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Car delta = new Car("Lancia", "Delta", 1984)
8         }
9     }
10
11     public class Car
12     {
13         public readonly string Brand;
14         public readonly string Model;
15         public readonly int Year;
16
17         public Car(string brand, string model, int year)
18         {
19             Brand = brand;
20             Model = model;
21             Year = year;
22         }
23     }
```

Diapositiva final

En este tema hemos aprendido conceptos teóricos avanzados sobre los tipos y las variables. Aunque en un principio puedan resultar abrumadores por su alto nivel de abstracción, es importante conocerlos para asentar las bases de cara a los siguientes temas.



Speaker notes